When Free Executors Cost More: The Free-Executor Paradox in Iterative LLM Code-Repair Loops

Ken Imoto
Propel-lab
Tokyo, Japan
imoto@propel-lab.co.jp
https://kenimoto.dev

June 2026

Abstract

The canonical recipe for cost-aware agentic coding—"use a strong model to orchestrate, a cheap model to execute"—fails on every iterative code-repair task we measured. We call this the *free-executor paradox*: when the executor is free at the token meter (a locally hosted Qwen 3.5-9B), the orchestrator's prompt-cached re-reads of the executor's returned summaries grow its own input volume by 1.4–5.3× over running Opus alone, and the structure becomes the **most expensive cloud configuration we tested**, not the cheapest. We support the claim with a controlled comparison of four configurations—Opus 4.7 solo, Opus orchestrating Qwen 3.5-9B (free executor), Opus orchestrating Haiku 4.5, and Haiku 4.5 solo—on three Python code-repair tasks (25-error breakage recovery, refactor, feature-add) over 40 trials with $n = 3$ successful runs per cell. Correctness is judged deterministically by `mypy + ruff + pytest` exit codes; no LLM grades the work. Beyond the paradox, we report two practitioner-level observations: (i) Haiku solo is 5.5× cheaper than the cheapest cloud arm on the longest task, but pays a 25% failure rate at our per-arm iteration cap; (ii) when Opus does orchestrate, Opus + Haiku—not the free executor—ties Opus solo on the longest task and beats it narrowly on the refactor task. Our headline question is therefore *not* "which model is best," but "under what task structure does the orchestrator's read-back cost amortize against the executor's per-token savings." We provide a harness, runners, and analysis that make this question reproducible on any codebase for about $36.

Artifact Code, raw trial data, figures, and the LaTeX source of this paper are open at `https://github.com/kenimo49/free-executor-paradox` (MIT code, CC-BY 4.0 text).
Zenodo DOI: *[reserved Zenodo DOI – replace via `scripts/insert-doi.sh`]*

1 Introduction

A standard cost-control move in agentic coding is to put a strong, expensive model in charge of planning and reasoning, and to delegate the mechanical edits to a cheap or free executor. The intuition is unambiguous: the executor handles the bulk of the tokens, and the orchestrator only pays for the decisions. We show that this intuition breaks down in the *iterative* regime that drives a deterministic judge to convergence, because the orchestrator is forced to re-read what the executor did on every subsequent iteration.

Most prior comparisons of orchestrated vs solo configurations are framed around *query-level* routing or single-shot cascades. RouteLLM [Ong et al., 2024] routes individual queries; FrugalGPT [Chen et al., 2023] cascades models on confidence. In both, the orchestrator's cost is paid once per user query. Agentic coding, by contrast, is iterative: the agent edits, runs the

harness, reads the failure, edits again, often 20–90 times per task. The orchestrator's input grows with the trajectory, not with the query. To our knowledge, that growth has not been quantified against a deterministic judge in published work.

The field has settled on two informal heuristics for the iterative regime—"biggest model, every call" and "one cheap model, more retries"—neither of which is justified by direct comparison under a fair judge. Most published comparisons of agentic coding rely either on LLM-as-judge or on aggregate benchmarks (HumanEval, MBPP, SWE-bench), and neither captures the cost-iteration-correctness tradeoff faced by an agent that *is allowed to retry until it converges*.

This paper makes three contributions:

1. We use a **decision-deterministic** harness—three tools (mypy, ruff, pytest) whose exit codes alone constitute the correctness judgment. No LLM grades the work. This removes one common confounder from agentic-coding benchmarks.

2. We add a "**$0 marginal cost executor**" axis to the orchestration comparison, in the form of a locally hosted Qwen 3.5-9B model.

3. We measure **iterations to convergence** as a primary metric alongside dollars and wall-time, giving budget-aware practitioners a directly actionable signal.

The empirical result of our small experiment is that the canonical "cheap-executor" orchestration form (Opus + Qwen, arm B) does not pay off on any of the three tasks we tested. The mechanism is straightforward: with prompt caching, the orchestrator's cost is dominated by its own context, which grows with executor output volume even when executor tokens themselves are free.

2 Related Work

Query-level routing. Ong et al. [2024] routes individual queries between strong and weak models based on a learned classifier. Our setting differs structurally: we measure the *full iterative loop* required to drive a deterministic judge to a green state, not a single query.

Cost-aware cascades. Chen et al. [2023] chains models in a cheap-to-expensive cascade, accepting an answer at the first stage with high enough confidence. We compare a cascade-adjacent setup (Opus orchestrator + cheap executor) under a hard convergence requirement rather than a confidence threshold.

Multi-LLM collaboration. Wang et al. [2024] aggregates outputs across multiple LLMs with LLM-judge synthesis. Our orchestrator-executor structure is functionally adjacent but uses a deterministic harness as judge.

Planner-executor agents. Shen et al. [2023] and Wang et al. [2023] use planner-executor decompositions but do not measure per-trial dollar cost against a deterministic correctness oracle.

Code-change benchmarks. For evaluation context, Jimenez et al. [2024] grade agentic code changes against real-world repository tests. Our setup is a much smaller, controlled variant in which the same harness is reused across configurations rather than a benchmark suite of unseen problems. To our knowledge, we are not aware of a prior published comparison that combines (i) Anthropic Opus vs Haiku on the same task harness, (ii) a locally hosted "free executor" axis, and (iii) a non-LLM judge.

3 Method

3.1 Arms

arm	orchestrator	executor	role split
A	Opus 4.7	(solo)	one model handles everything
B	Opus 4.7	Qwen 3.5-9B (local, Ollama)	Opus plans + verifies, Qwen edits
C	Opus 4.7	Haiku 4.5	Opus plans + verifies, Haiku edits
D	Haiku 4.5	(solo)	one cheap model handles everything

Table 1: Four configurations under comparison. All call the Anthropic API through the SDK; arm B additionally calls Qwen via the `qwen-task -agent` wrapper.

All four arms have access to identical tools: a `str_replace_editor` (view / create / str_replace / insert) scoped to the typer repository root, and a `bash` tool with a 120-second timeout. Orchestrators (B, C) additionally have a `delegate_to_executor(instruction)` tool that hands a single concrete instruction string to the executor.

The orchestrator and executor see **slightly different system prompts**. The solo system prompt (A, D) lets the model freely use `str_replace_editor`. The orchestrator prompt asks the model to avoid editing directly and to delegate edits instead. This asymmetry is intentional—it is how a real orchestrator/executor split would be deployed—but it is also a confounder, since it shapes the orchestrator's behavior beyond just adding the delegate tool. We return to this in Section 6.

Anthropic prompt caching is enabled identically on every call: `system`, tool definitions, and the most recent user message are marked with `cache_control: ephemeral`. No `temperature` or `seed` is set on either Anthropic or Qwen calls, so trial-to-trial variance reflects sampling stochasticity.

3.2 Tasks

All three tasks operate on the `typer` [Ramírez and contributors, 2024] repository at tag `0.26.8` (commit `b210c0e`, MIT license). Each trial begins with `git checkout -- . && git clean -fd typer/ tests/` to restore the base state. The base repository is cloned at experiment time, not vendored.

T1 — Breakage recovery. A Python script using `ast.parse` deterministically injects 25 errors across 15 source / test files: 10 mypy type errors, 10 ruff lint errors, and 5 pytest collection-time failures (`assert False` lines prepended after the last top-level import in five test modules). The agent must return the harness to fully green.

T2 — Refactor. Move the function `get_params_from_function` from `typer/utils.py` to a new module `typer/_param_extractor.py`. Update every import site so that no file imports the function from the old location and at least one imports it from the new location. All tests must continue to pass.

T3 — Feature-add. Implement `get_version_banner(prefix: str = "Typer", uppercase: bool = False) -> str`, re-export it from `typer/__init__.py`, and make it pass a provided test file. The test file is fingerprinted with SHA-256 at injection time; the verifier rejects any modification to it. We acknowledge that this fingerprinting biases T3 toward "specification-following" behavior over exploration.

3.3 Harness

A bash script runs `uv run mypy`, `uv run ruff check`, `uv run ruff format` (informational only, not a failure condition), and `uv run pytest` in sequence and emits a single JSON blob with per-tool counts. A green base on this harness takes 22 seconds and reports 1356 tests passing. T3 adds 3 expected passes. `verify-T2.sh` and `verify-T3.sh` add task-specific structural checks (e.g. function moved to new module, public API callable, fingerprinted test unchanged).

3.4 Metrics

Per trial, we record:

- **success** — verifier exit code = 0,
- **iterations** — outer turns of the tool loop on the orchestrator side,
- **wall_sec** — seconds from `setup_task` to verifier completion,
- **cost_usd** — tokens × public rate card (Opus 4.7: \$15 / \$75 per M input / output; Haiku 4.5: \$1 / \$5; cache write 1.25×, cache read 0.10× of the input rate; Qwen marginal cost \$0),
- per-role token usage (input / output / cache_write / cache_read).

3.5 Trial budget

Per-arm outer-loop caps are a design variable, not a model property:

arm	T1 cap	T2 cap	T3 cap
A	40	40	40
B	80	60	60
C	40	40	40
D	100	120	80

Table 2: Iteration caps per arm and task. D (Haiku solo) needs the highest cap because its per-iteration capability is lowest.

Within these caps, failures count against the cell's success rate rather than being silently retried. We aim for $n \geq 3$ *successful* trials per (arm, task) cell. Trials that exhaust the cap are recorded as failures and excluded from median statistics; their counts contribute to the visible success-rate column.

3.6 Statistical procedure

We report median and IQR per cell (success-only), and pairwise Mann-Whitney U tests on the `cost_usd` column within each task. Two-sided p-values use a normal approximation; under that approximation and $n = 3$–4 per cell, $p = 0.050$ is the boundary value. An exact Mann-Whitney U would yield slightly different small-sample p; we read $p = 0.050$ here as "as different as this sample size can show under this approximation" rather than as a strong significance claim.

4 Results

4.1 Summary table

Table 3 reports per-cell success-only medians, the 95% bootstrap confidence interval for cost, and a *cost-per-success* column (median cost divided by success rate) that approximates expected cost under a retry-on-failure policy.

arm	task	n_s/n	wall (s)	iters	cost ($)	95% CI	cost/succ	success
A Opus solo	T1	3 / 3	253	36	1.74	[1.24, 1.84]	1.74	1.00
A Opus solo	T2	3 / 4	233	26	1.11	[0.86, 1.14]	1.48	0.75
A Opus solo	T3	3 / 3	**69**	**6**	0.17	[0.17, 0.43]	0.17	1.00
B Opus+Qwen	T1	3 / 4	484	38	2.27	[2.05, 2.60]	3.03	0.75
B Opus+Qwen	T2	3 / 3	443	27	1.38	[1.20, 1.48]	1.38	1.00
B Opus+Qwen	T3	3 / 3	348	12	0.42	[0.42, 0.70]	0.42	1.00
C Opus+Haiku	T1	3 / 3	400	**28**	1.67	[1.08, 2.07]	1.67	1.00
C Opus+Haiku	T2	3 / 3	275	**20**	**0.92**†	[0.91, 1.43]	0.92	1.00
C Opus+Haiku	T3	3 / 3	145	11	0.38	[0.35, 0.43]	0.38	1.00
D Haiku solo	T1	3 / 4	758	89	**0.30**	[0.19, 0.43]	0.40	0.75
D Haiku solo	T2	3 / 4	507	70	**0.23**	[0.23, 0.31]	0.31	0.75
D Haiku solo	T3	3 / 3	208	29	**0.08**	[0.07, 0.08]	0.08	1.00

Table 3: Success-only medians, bootstrap 95% CI (10,000 resamples), and cost-per-success. Bolded numbers are per-task per-column winners. † C is the cheapest *cloud* arm on T2; D is cheaper overall but with a 25% failure rate. Cell totals: 40 trials, total Anthropic spend $35.98.

4.2 Pareto frontiers

Figures 1, 2, and 3 show cost (USD) on the x-axis and wall-time (seconds) on the y-axis per task. Each arm contributes a colored cluster of trial dots and a square marker at the success-only median.

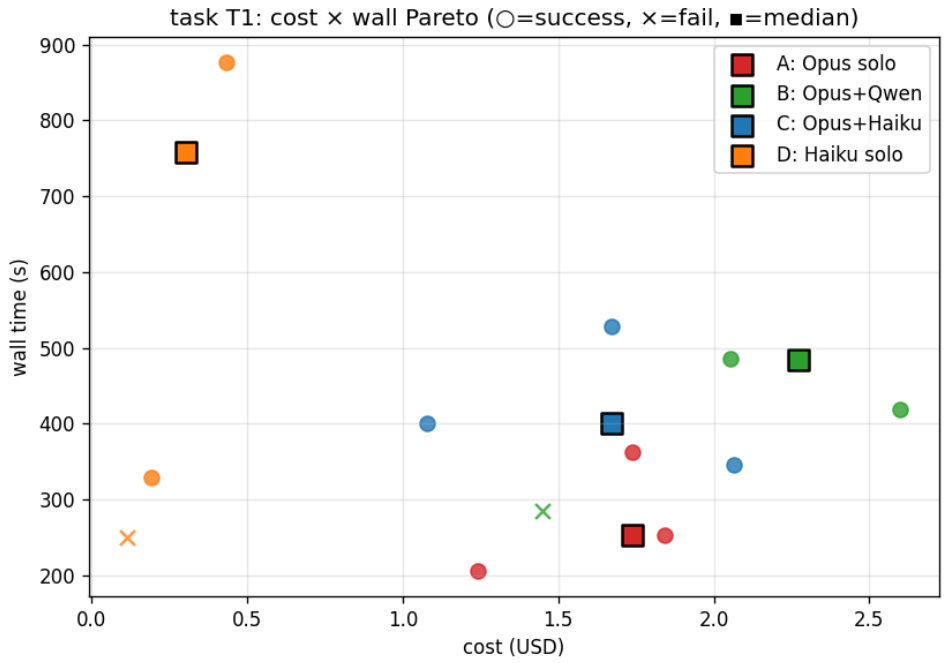

Figure 1: T1 breakage-recovery Pareto. D holds the cost extreme; C takes over for cloud-only setups; A holds the wall-time extreme; B is dominated on both axes by C. One D, one A T2, one B T1, and one D T2 trial fail at the per-arm cap (shown as ×).

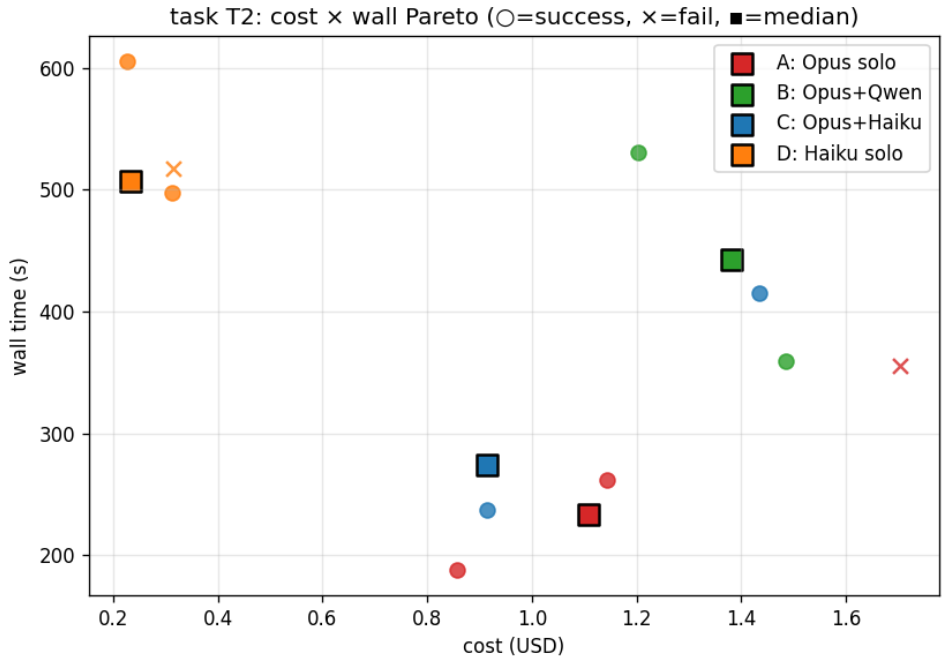

Figure 2: T2 refactor Pareto. D and C anchor the frontier; A is on the frontier (faster than C); B is dominated.

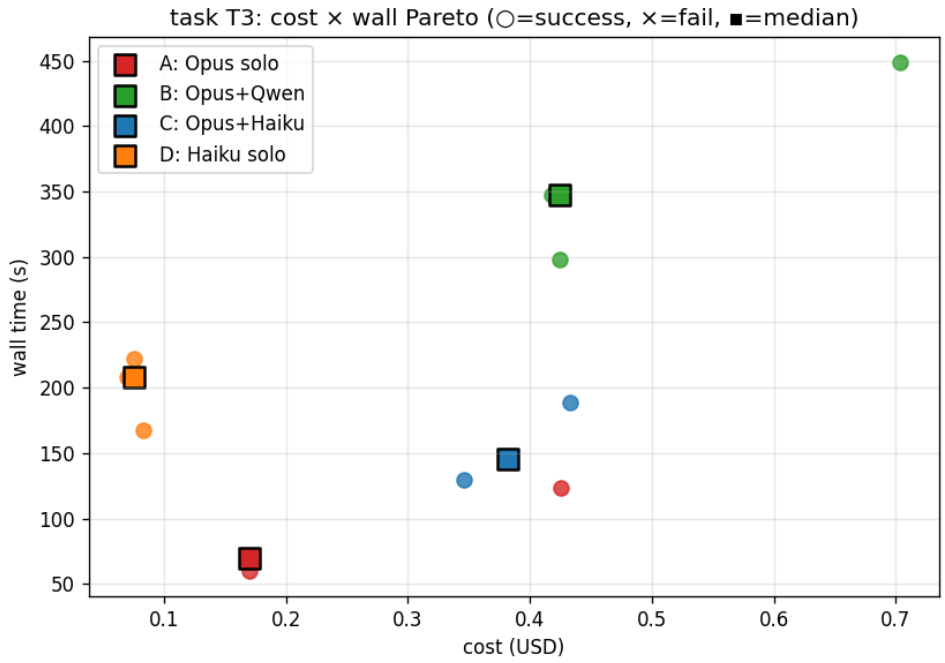

Figure 3: T3 feature-add Pareto. Opus solo (A) at \$0.17 / 69 s is Pareto-dominant on wall-time and undominated except by Haiku solo on cost. B and C are both dominated by A on this task.

4.3 Pairwise Mann-Whitney U (cost) per task

task	A vs B	A vs C	A vs D	B vs C	B vs D	C vs D
T1	0.050	0.827	0.050	0.127	0.050	0.050
T2	0.050	0.827	0.050	0.275	0.050	0.050
T3	0.275	0.275	0.050	0.275	0.050	0.050

Table 4: Two-sided p-values, normal approximation. Under this approximation and $n = 3$–4, $p = 0.050$ is the boundary value.

Under this approximation, we did not detect a cost difference between A and C on T1 and T2 ($p = 0.827$). D's cost is at the minimum observable p ($= 0.050$) against every other arm on every task, but the small sample size limits the strength of this claim. B (Opus + Qwen) is at the same boundary p versus A on T1 and T2, despite Qwen's tokens being free at the meter.

To complement Mann-Whitney U under small n, Table 5 reports Cliff's delta [Romano et al., 2006] on the cost column. The direction is read as "how often does the row arm cost less than the column arm"—negative means row is cheaper. All comparisons of B against the cheaper arms (A on T1/T2, C, D) yield a *large* positive delta, confirming that the rank ordering is not an artifact of one outlier trial.

task	B vs A	B vs C	C vs A	D vs C
T1	+1.00 (large)	+0.78 (large)	−0.11 (negligible)	−1.00 (large)
T2	+1.00 (large)	+0.56 (large)	+0.11 (negligible)	−1.00 (large)
T3	+1.00 (large)	−0.11 (neg.)	+1.00 (large)	−1.00 (large)

Table 5: Cliff's delta on cost (positive = row arm is more expensive than column arm). Effect-size labels follow Romano et al. [2006]: $|d| < 0.147$ negligible, < 0.330 small, < 0.474 medium, otherwise large.

4.4 The free-executor paradox

Compare the per-arm Opus-side token volume `input + cache_read_input` across tasks (success-only median):

arm role (Opus side)	T1	T2	T3
A (Opus solo)	534,586	226,474	13,320
B (Opus orchestrating Qwen)	733,142	313,914	62,864
C (Opus orchestrating Haiku)	421,622	159,640	44,016

Table 6: Opus input + cache_read tokens, success-only median.

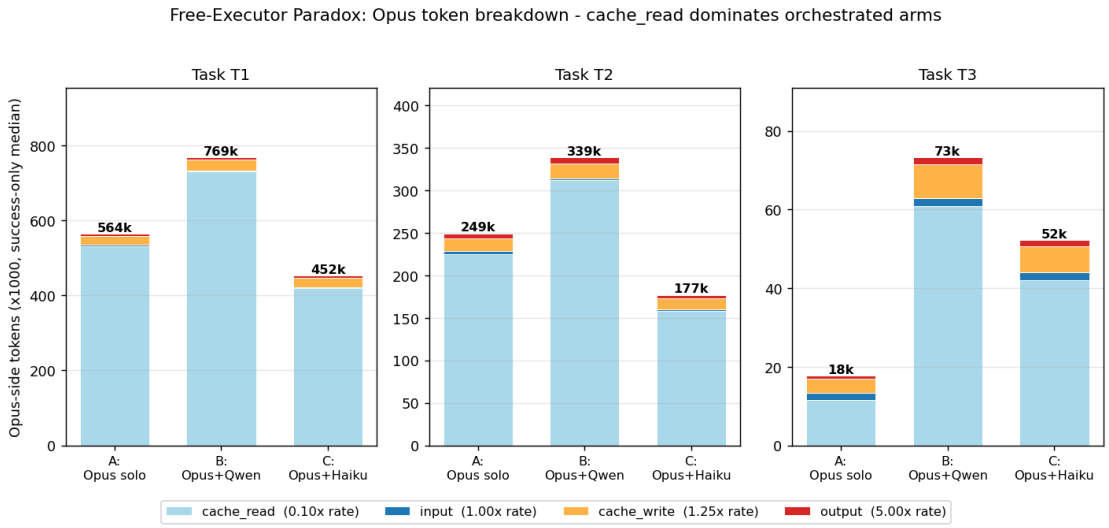

Figure 4: Free-executor paradox visualized as a token-component breakdown. Each stacked bar is the success-only median of Opus-side tokens for one (task, arm) cell, decomposed into `cache_read` (light blue, billed at 0.10× input rate), `input` (dark blue, 1.00×), `cache_write` (orange, 1.25×), and `output` (red, 5.00×). Two patterns: (i) the dominant component is `cache_read` (orchestrator re-reading prior turns), not new input or output; (ii) arm B's bars are the tallest in every task, even though B's executor (Qwen) costs $0 at the token meter.

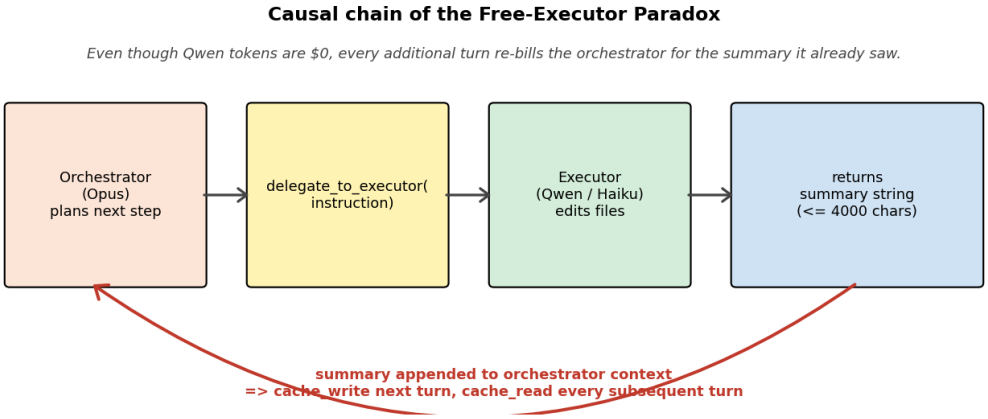

Figure 5: Causal chain underlying the paradox. The orchestrator delegates an instruction, the executor edits files and returns a summary string, and that summary is appended to the orchestrator's context. On the *next* turn that summary becomes a `cache_write`; on every subsequent turn it becomes part of the `cache_read` payload, re-billed at 10% of the input rate per iteration. Over 30–80 iterations of T1 this loop pays for the entire executor delegation many times over.

The B-over-A ratio of `cache_read` alone is **1.38× on T1, 1.39× on T2, and 5.26× on T3**. Even though B's executor (Qwen) is free, the orchestrator side of B reads back more cache tokens per iteration than Opus solo. Two observations follow.

First, the multiplier is largest on T3, the smallest task. The base context (system + tools + initial prompt) is amortized across iterations; on long tasks (T1, T2) it is a smaller fraction of total input, so the orchestration overhead is less visible. On short tasks, the overhead dominates.

Second, C (Opus + Haiku) sees a *smaller* `cache_read` footprint than A (Opus solo) on T1 and T2 (0.79× and 0.70× of A). The executor in C does substantive work that Opus would otherwise have to do itself, and the executor's *summary* (not its full output) is what flows back into Opus.

The mechanism is therefore not "executor outputs free → orchestrator reads them all." It is "executor outputs summary → orchestrator reads summary, and on small tasks the summary is large relative to the base." This is consistent with our observed cost ordering: B is most expensive on T3 (where the multiplier is 5.3×), and only modestly more expensive than A on T1 (where the multiplier is 1.4×).

5 Discussion

5.1 Why orchestration loses on dollars in our setup

In a one-shot routing setup such as RouteLLM, the orchestrator's only cost is the routing decision; the chosen model bears the answer-generation cost alone. In an iterative tool loop, however, the orchestrator must read whatever the executor returns through its `delegate_to_executor` tool—in our implementation, that is a truncated stdout summary (Qwen, up to 4000 chars) or the executor's final text (Haiku, up to 1500 chars), not the executor's full intermediate output. Even so, that returned summary accumulates in the orchestrator's context across 30–80 iterations on T1 and feeds into `cache_write` and `cache_read` on every subsequent turn.

A free executor's "free" applies only to that executor's own tokens. The orchestrator's cost grows in proportion to how much it must *re-read* about what the executor did, not how much the executor itself produced.

5.2 When orchestration would win

Our setup does not reproduce the regimes in which orchestration is most likely to pay:

- Executors whose returned summaries are tightly bounded—e.g. one structured diff, not free-form prose. Our `qwen-task -agent` wrapper does not enforce that.
- Tasks with many independent sub-problems that can run in parallel under one orchestration. Our three tasks are inherently sequential within a single trial.
- Repeated trials in which the orchestrator can amortize its cache across runs. We reset the harness between trials, so each trial pays its own `cache_write`.

A reasonable next experiment would constrain executor return size to a few hundred tokens per delegation and rerun arm B; we expect the gap to A to narrow, and perhaps invert on T3 specifically.

5.3 The cost-reliability tradeoff

The cost ordering D $\ll$ C $\approx$ A $<$ B is qualified by reliability. D's per-trial cost advantage on T1 ($0.30 vs C $1.67, ratio 5.5×) is bought at a 25% failure rate within our per-arm iteration cap. For a CI pipeline that retries failed runs, the expected cost is roughly $cost\_med/success\_rate$; D's *expected* T1 cost becomes $0.40, narrowing the cloud-vs-Haiku gap from 5.5× to 4.2×. For a human in the loop who pays for failed runs as time, the gap is wider.

The right model choice is a function of the cost of failure recovery, not just of median cost per trial.

5.4 Performance differs across task types

T3 (feature-add) requires about 6 iterations of edit-test-edit. Within that small budget, Opus's per-iteration efficiency pays for itself: A is undominated on wall-time, iterations, and cloud-only cost simultaneously. T1 (25-error recovery) requires 30–90 iterations regardless of model; here Haiku's lower per-iteration cost dominates the dollar axis, and C—sharing cheap iteration-time costs between Opus and Haiku—is the cheapest cloud-only arm. T2 sits between the two on iteration count, and C narrowly wins the cloud-only cost axis ($0.92 vs A's $1.11).

These three task types are not points along a single "complexity" axis we have measured. They differ in iteration count, in the structural specificity of the verifier, and in how much the agent is rewarded for spec-reading versus exploration. We describe the result as "varies across task types" rather than as "rotates with complexity."

The practical guidance we are willing to give is narrower: **use the cheaper-per-iteration model when the task requires many iterations, and consider orchestration when the executor's returned summary can be tightly bounded**—neither of which our arm B reliably delivers.

6 Threats to Validity

We organize threats to validity along the standard software-engineering empirical-research axes [Wohlin et al., 2012]: internal validity (do the results follow from the experiment as designed), external validity (do they generalize beyond it), and construct validity (does the measurement capture what we claim).

6.1 Internal Validity

- **Sampling non-determinism.** We did not pin `temperature` or `seed` on any Anthropic or Qwen call. Reported variance is sampling-driven and partly an artifact of API defaults. Within-cell variance is non-trivial: A T3 iterations were 6 / 6 / 16, and C T2 iterations were 17 / 20 / 36.
- **Iteration cap as design variable.** The per-arm cap (Section 3.5) is set by the experimenter. Failures within budget would become successes given a larger cap. The success-rate column reflects cap policy, not pure model capability.
- **System-prompt asymmetry between solo and orchestrator arms.** A and D's prompts let them edit freely; B and C's orchestrator prompts ask them to avoid direct edits and delegate instead. This is faithful to deployment but is a confounder for any "structure alone" comparison.
- **Statistical strength.** With $n = 3$ successful trials per cell, the minimum two-sided Mann-Whitney p under normal approximation is 0.050. The Cliff's-delta complement (Table 5) helps but does not replace larger n.

6.2 External Validity

- **Single repository.** All three tasks are scoped within `typer` v0.26.8. Whether the free-executor paradox extends to larger codebases (e.g. `httpx`, `click`, `fastapi`) is future work. We expect the qualitative direction to hold because the mechanism (orchestrator re-reads of executor summaries through prompt cache) is repository-independent, but the magnitude should grow with longer task trajectories.
- **Two cloud models.** We compare Anthropic Opus 4.7 and Haiku 4.5 only. Sonnet, GPT-class models, and Gemini are absent.
- **One local executor.** Qwen 3.5-9B is the only local model tested. A 30B-class local model would shift wall-time and possibly executor capability, but not dollar cost.
- **No parallel sub-task structure.** Our three tasks are sequential within a single trial. We do not measure regimes where multiple delegate calls could fan out concurrently, which would shift the orchestrator's amortization profile.

6.3 Construct Validity

- **Test fingerprinting (T3).** SHA-256 fingerprinting of the T3 test file biases T3 toward spec-following behavior over exploration. The conclusion "Opus solo dominates on T3" should be read as "on spec-following tasks of T3's size," not as "on all small tasks."
- **The harness itself.** `mypy + ruff + pytest` exit code = 0 is one correctness oracle. Code-review quality, security, and maintainability are not measured here.
- **Executor summary as proxy for executor output.** The orchestrator never sees the executor's raw work; it sees a truncated summary returned by the `delegate_to_executor` tool. This is how real orchestrator-executor systems are deployed, but it means our paradox quantifies summary read-back, not raw output transfer.

6.4 Threat Matrix

Table 7 summarizes the threats above. The **Impact** column is a subjective rating of how much each threat could change the paper's headline claim (the free-executor paradox itself, the cost ordering, and the practical guidance), color-coded High, Med, Low.

Threat	Type	Impact	Headline effect	Mitigation in place / proposed
Prompt asymmetry (solo vs orch.)	Internal	**High**	cross-arm comparability	documented (§3.5); future: symmetric ablation
Single repository	External	**High**	generalization beyond `typer`	mechanism is repo-independent (§5.1); future: `httpx`, `click`, `fastapi`
Sampling nondeterminism	Internal	**Med**	within-cell variance	95% bootstrap CI (Table 3); future: pin temperature
Iteration cap	Internal	**Med**	success-rate column	cost-per-success and CI; future: $n \times 2$ at next cap
Small n ($= 3$)	Internal	**Med**	p-value floor at 0.050	Cliff's delta (Table 5); future: $n \to$ 10–20
Two cloud models	External	**Med**	vendor specificity	structural argument is vendor-agnostic; future: GPT-5, Gemini 2
No parallel delegation	External	**Med**	amortization regime	sequential structure documented; future: parallel-fanout extension
T3 SHA-256 fingerprint	Construct	**Med**	T3 specifically	T3 claim limited to spec-following; future: open-spec variant
Summary as proxy for output	Construct	**Med**	mechanism scope	we measure summary read-back; future: enforce summary $\leq$ 500 tokens
One local executor	External	**Low**	executor-LLM specificity	paradox depends on summary volume, not on Qwen; future: 30B-class local
Harness as oracle	Construct	**Low**	code-quality unmeasured	we claim correctness only, not maintainability; future: LLM-judge complement

Table 7: Threat matrix sorted by Impact then Type. "Mitigation in place" refers to the current paper; "proposed" is deferred to future work (§7). The two High-impact threats—prompt asymmetry and single-repository scope—are also the two most concrete next experiments.

7 Future Work

The mechanism we report has at least four extensions that we view as the most useful next experiments, in roughly increasing cost order.

1. Bounded-summary executors. The orchestrator's cache_$read footprint grows with the size of the executor's$ $form$ `stdout` $returns with structured summaries capped at a few hundred tokens (e.g. a JSON diff, a unified-format patch, or a single sentence) should shrink the B/A ratio toward 1.0\times$ and potentially invert it on T3. We expect this to be the cheapest experiment to run on the existing harness.

2. Larger repositories. Replicating the 4-arm × 3-task design on `httpx`, `click`, and `fastapi` would address the external-validity gap directly. The mechanism predicts the paradox should hold and that the dollar gap should *widen* on longer trajectories, because cache_read is the dominant term. A counter-finding (no paradox on `fastapi`, say) would be diagnostic of repository structure, not of the mechanism.

3. Additional model families. Adding Sonnet 4.6 (between Opus and Haiku in capability and cost) tests whether the paradox is a property of strong-vs-weak pairing in general or specifically of Opus–Haiku. GPT-5 and Gemini 2 (when stable cache pricing is published) extend the test across vendors. We expect the qualitative pattern to hold because prompt caching has converged across vendors, but the precise multipliers depend on rate-card structure.

4. Parallel orchestration. The current design is strictly sequential within a trial. A parallel delegation regime—multiple `delegate_to_executor` calls fanned out per orchestrator turn—would shift the orchestrator's amortization profile, because the orchestrator pays the cache cost *once* for k executor calls instead of k times. This is the most architecturally different extension and the one most likely to surface a regime where arm B wins.

We also intend to mirror the harness on `cargo check + cargo test` for Rust and on `go vet + go test` for Go, to verify the mechanism is not a Python-specific artifact.

8 Conclusion

We did not set out to declare a winner. We set out to ask whether the cost gap between "use Opus everywhere" and "use Opus to orchestrate something cheap" is large enough to justify the extra plumbing.

For our particular cheap-executor instantiation—Opus orchestrating local Qwen 3.5-9B (arm B)—the answer is *no on every task we tested*. The mechanism is straightforward: prompt-cached orchestration re-reads the executor's returned summary on every iteration, so an executor that is "free" at the token meter is not free at the orchestrator's input.

The picture is more nuanced for Opus + Haiku (arm C), which beats Opus solo on the cloud-only dollar axis for T2 and ties it for T1. Within the cloud, "orchestrate when the executor is also a competent reasoner" is a defensible heuristic; "orchestrate so the executor can be free" is not in our setup.

The contribution we hope is most reusable is the methodology, not the verdict. A harness-judged comparison cost about $36 of API time and a few hours of wall-clock to produce twelve cells of Pareto data on a real codebase. The numbers should be small enough for practitioners to run their own comparisons on their own codebases.

Reproducibility

Tested on Ubuntu 22.04 with Python 3.10+ (typer requires `>=3.10`), `uv` 0.4+, and `anthropic` Python SDK 0.83+. The Qwen executor uses Ollama 0.4+ on a Tailscale-reachable host running model `qwen3.5:9b`, exposed via the `qwen-task -agent` wrapper bundled at `scripts/qwen-task.sh` (the wrapper itself depends on the `claw` binary; see README for setup).

```
# 1. Clone this repo
git clone https://github.com/kenimo49/free-executor-paradox
cd free-executor-paradox

# 2. Clone base repo (typer at 0.26.8, commit b210c0e)
git clone --depth 1 --branch 0.26.8 \
```

```
    https://github.com/tiangolo/typer base-repo/typer
(cd base-repo/typer && uv sync)

# 3. Confirm green base on the harness
./harness.sh --json-only   # expect exit_code=0, 1356 passed

# 4. Provide ANTHROPIC_API_KEY (env or <repo>/.env)
export ANTHROPIC_API_KEY=...

# 5. (Optional) point at Ollama host for arm B
export OLLAMA_HOST=http://<tailscale-host>:11434

# 6. Run a single trial end-to-end
python -m runners.runner --arm A --task T3 --trial 0

# 7. Full re-run targeting n=3 successes/cell
./runners/batch.sh all 3

# 8. Regenerate tables + Pareto plots
python3 analysis/analyze.py --plot
python3 analysis/export_csv.py
```

For arm B specifically: if Ollama / Tailscale are unavailable, arm B trials will time out at the delegate-tool boundary; arms A, C, D remain runnable.

Acknowledgements

The `typer` project (Sebastián Ramírez et al., MIT) provided the substrate. The Qwen team open-sourced the model used in arm B. The Codex CLI from OpenAI was used to perform an independent read-only review of this manuscript prior to revision; review records are linked from the GitHub issue tracker.

References

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

Sebastián Ramírez and contributors. Typer: build great CLIs. easy to code. based on Python type hints. `https://github.com/tiangolo/typer`, 2024. Version 0.26.8, commit b210c0e, MIT License.

Jeanine Romano, Jeffrey D. Kromrey, Jesse Coraggio, and Jeff Skowronek. Appropriate statistics for ordinal level data: Should we really be using t-test and cohen's d for evaluating group differences on the NSSE and other surveys? In *Annual Meeting of the Florida Association of Institutional Research*, pages 1–33, 2006.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. *arXiv preprint arXiv:2303.17580*, 2023.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024.

Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012. doi: 10.1007/978-3-642-29044-2.

A System prompts

The four arms use prompts of the same surface form. We render them as a side-by-side diff to make the asymmetry explicit; it is a confounder we discuss in §6.

Legend. Red highlight = solo-only text (arms A and D). Green highlight = orchestrator-only text (arms B and C). Unhighlighted text is shared verbatim between both prompts.

A.1 Solo (arms A, D) vs orchestrator (arms B, C) — unified diff

```
You are a code-fixing agent the orchestrator of a two-LLM team. You decide WHAT to
do; an executor LLM (which can read files, edit files, and run shell commands) does
it for you.

operating in a sandboxed Python repository.

You have two three tools:

• bash:  run shell commands in the repo root (120s timeout).
• str_replace_editor:  view / create / str_replace / insert files (paths are repo-
relative). read files yourself with ‘view‘ when you need to inspect something
before instructing the executor. Avoid using create/str_replace -- delegate edits
to the executor.
• delegate_to_executor: hand off a concrete edit/refactor instruction to the execu
tor. The executor returns a short summary of what it did. Be precise: name files,
name functions, name the exact change. The executor is cheaper but weaker than you,
so spell out the change rather than asking it to "figure it out".

You will be given a task.  Iteratively read code, run the harness, fix issues,
and re-run until the harness is green.  Be concise in your reasoning; spend tokens
on tool calls, not narration. Loop: read harness output -> plan -> delegate small
chunks -> check harness -> repeat until green.

When you believe the task is complete and the verifier passes, output a single line
‘DONE’ and stop.  If you cannot make progress, output ‘BLOCKED: <reason>’ and stop.
```

What the diff implies. The orchestrator prompt explicitly asks the model to *avoid editing directly*; the solo prompt does not constrain editing. This is a fair representation of how the structures are deployed in practice, but it is a confounder for any "structure alone" comparison: arm A/D may be advantaged by full editing freedom independent of model capability. Resolving this with a symmetric ablation is item 1 of our internal-validity future work (§7).

A.2 Executor prompt (arm C only; arm B uses qwen-task default)

```
You are an execution sub-agent. Another LLM (the orchestrator) has
handed you a concrete instruction. Do exactly what it asks using
‘str_replace_editor‘ and ‘bash‘. Do not expand scope. When done, output
‘DONE: <one-line summary>‘ and stop. If the instruction is impossible
as written, output ‘BLOCKED: <reason>‘ and stop.
```

B Harness output schema (sample)

`harness.sh -json-only` emits the following structure on a green base (`exit_code = 0`):

```
{
  "exit_code": 0,
  "elapsed_sec": 22,
  "mypy":         {"exit": 0, "errors": 0},
  "ruff_check":   {"exit": 0, "errors": 0},
  "ruff_format":  {"exit": 0, "errors": 0, "informational": true},
  "pytest":       {"exit": 0, "passed": 1356, "failed": 0,
                   "collection_errors": 0}
}
```

`exit_code` is the only field the runner consumes for the gate (0 = pass). Per-tool counts are recorded for diagnostics and end up in the per-trial JSONL.

`verify-T2.sh` and `verify-T3.sh` wrap `harness.sh` and add task-specific structural checks (function moved to new module, public API callable, fingerprinted test unchanged); they emit the same outer schema with an extra `t2_checks` or `t3_checks` block.

C Per-trial JSONL schema

Each trial appends one JSON object to `data/results/{arm}.jsonl`:

```
{
  "arm": "C", "task": "T1", "trial": 0,
  "success": true, "blocked": false,
  "wall_sec": 400.4, "iterations": 18,
  "usage_by_role": {
    "claude-opus-4-7": {
      "input_tokens": 2050, "output_tokens": 3803,
      "cache_creation_input_tokens": 17089,
      "cache_read_input_tokens": 170101
    },
    "claude-haiku-4-5-20251001": {
      "input_tokens": 15765, "output_tokens": 9774,
      "cache_creation_input_tokens": 62382,
      "cache_read_input_tokens": 435940
    }
  },
  "cost_usd_by_role": {
    "claude-opus-4-7": 0.891545,
    "claude-haiku-4-5-20251001": 0.186206
  },
```

```
  "total_cost_usd": 1.077752,
  "verify_final": {
    "exit_code": 0,
    "summary": "exit_code=0, mypy errors=0, ruff errors=0, "
               "pytest passed=1356 failed=0 collection_errors=0"
  }
}
```


The 40 trials reported in this paper live in `data/results/{A,B,C,D}.jsonl` and the flattened table is at `data/trials.csv`.